

Caching

A system design guide: how caching makes systems faster, cheaper, and more reliable

Section 1: Introduction to Caching

Caching is a technique used to improve the efficiency and performance of a system by storing a copy of data in a temporary storage layer (the **cache**).

The idea is simple: instead of repeating expensive work (a slow database query, a heavy computation, a request to a faraway server) every single time, the system does the work **once**, keeps the result in fast storage (usually RAM), and serves that copy for the next requests.

Two things to always remember: the cache is **fast but small and temporary**, and it is **never the source of truth**: the real data always lives in the database.

Section 2: Benefits of Caching

- **Reduced Latency**: the data is served from fast memory close to the user instead of a slow database or a faraway server, so responses arrive in milliseconds.
- **Lowered Server Load**: repeated requests are answered by the cache, so the database and servers handle far fewer queries. This also protects the system during traffic spikes.
- **Improved UX**: pages load faster and the app feels instant, and faster experiences keep users engaged.

Section 3: Places Cache Can Be Stored

A cache is not one single thing. Copies of data can live at several layers, each one closer to or further from the user. A request travels from left to right, and each layer can answer it before it ever reaches the database:

Browser Cache	CDN	Server Cache	Database Cache	Database
on the user's device	nearest edge server	e.g. Redis	inside the DB engine	source of truth

Browser Caching

- The user's own browser stores copies of **static files** (images, CSS, JavaScript, fonts) on the user's device the first time they visit a site.

- On the next visits, those files load instantly from the device, with **zero network requests**.
- It is controlled by HTTP headers such as **Cache-Control**, **Expires**, and **ETag**, which tell the browser what to store and for how long.
- One catch: you can't easily clear a user's browser cache from your side. That is why sites rename files when they change (style.v2.css), forcing browsers to download the new version. This trick is called **cache busting**.

Server Caching

- The cache is stored in the server's memory, on an external cache server like **Redis** (or Memcached), or on disk.
- The server **checks the cache** for the data before querying the database.
- If the data is in the cache (a **hit**) it is returned directly. If it isn't found (a **miss**), the server queries the database, stores the result in the cache for future requests, then returns it.
- This check-then-fill pattern is known as **cache-aside (lazy loading)**: the cache only fills up with data that is actually being requested.

Database Caching

- The database itself keeps its most frequently accessed data **in memory**: recently read rows, index pages, and query results.
- Most databases (MySQL, PostgreSQL, MongoDB) do this **automatically** in what is called the **buffer pool**, with no code needed from you.
- It makes repeated reads much faster because they come from RAM instead of disk.
- Its limit: it only speeds up work **inside the database machine**. It does not remove the network trip from your server to the database the way Redis does.

CDNs (Content Delivery Networks)

- A network of servers scattered geographically which cache the data from the original (origin) server and deliver it from the CDN server **nearest to the user**.
- CDNs are covered in depth in Sections 7, 8, and 9.

Section 4: Cache Ratio

- **Cache Hit**: the requested data is found in the cache.
- **Cache Miss**: the requested data isn't found in the cache and needs to be fetched from the original source (most times the database). A miss is the slowest case, because you pay for the cache check and the database query.

$$\text{Cache Hit Ratio} = \text{Cache Hits} / (\text{Cache Hits} + \text{Cache Misses})$$

- **Example**: 900 hits and 100 misses give $900 / 1,000 = 0.9$, which is a 90% hit ratio.

- **Why it matters:** this is the number one metric for judging a cache. A high ratio (90%+) means the cache is doing its job. A low ratio means the cache is too small, items expire too quickly, or the wrong data is being cached.

Section 5: Write-Around vs Write-Through vs Write-Back Cache

Reading from a cache is easy. The real design decision is what happens when data is **written**. There are three classic strategies:

Write-Around Cache

- Data is written **directly to permanent storage**, bypassing the cache. The cache only gets the data later, when someone reads it.
- It is used when write performance is less critical and new data is not read immediately.

Write-Through Cache

- Data is **simultaneously written to the cache and to permanent storage**.
- It ensures **data consistency** (the cache and the database never disagree), but it may be slower than write-around because every write is done twice.

Write-Back Cache

- Data is first **written to the cache**, then to permanent storage at a later time, in the background.
- This greatly **improves write performance**, but has a risk of losing that data in case of a server crash before it is saved.

At a glance:

Strategy	How writes happen	Best for	Trade-off
Write-Around	Write goes straight to the database; the cache is skipped and only fills when the data is read later	Data that is written but rarely read right away (logs, archives)	First read after a write is always a cache miss
Write-Through	Write goes to the cache and the database at the same time	Data that must always be consistent and is read right after writing	Every write is done twice, so writes are slower
Write-Back	Write goes to the cache first; the database is updated later in the background	Very write-heavy systems (counters, analytics)	If the cache crashes before saving, that data is lost

Section 6: Eviction Policies

An eviction policy is used when the cache is **full** and we need to free up some space so it can be used again. Eviction policies are the rules that determine which items get removed from the cache when it's full.

The 3 most used policies:

- **LRU (Least Recently Used):** removes the item that hasn't been used for the longest time. The logic: if you haven't touched it in a while, you probably won't need it soon. This is the most popular policy in practice.
- **FIFO (First In First Out):** removes the oldest item added to the cache, no matter how often it is used. Very simple, but it can throw away items that are still popular.
- **LFU (Least Frequently Used):** removes the item that was used the fewest times. Great when popularity is stable, but it needs a counter for every item and can hold on to data that was popular once but isn't anymore.

Related tip: separate from eviction, most caches also give every item a **TTL (Time To Live)**: an expiry timer so old data disappears automatically even when the cache is not full. Eviction manages space; TTL manages freshness.

Section 7: More About CDNs

The two types of CDNs are:

Pull-Based CDN

- The user requests data from the nearest CDN server. If the data isn't found on that CDN server, it requests (pulls) it from the original server, stores a copy, then sends it to the user.
- The **first** user gets a slower response, but everyone after them is served instantly from the CDN.
- **Low maintenance:** the CDN fills itself automatically based on what users actually request.

Push-Based CDN

- You upload the data or files to the original server, then **distribute (push)** them to the other CDN servers yourself, before anyone asks for them.
- It is useful when we have **large files** that don't need constant updating but need to be quickly redistributed when updated (software releases, game files, videos).
- Requires **more frequent management**: you decide what gets pushed, where, and when.

Section 8: Benefits of Using CDNs

- **Reduced Latency:** content is delivered from a server physically close to the user, so it travels a shorter distance and arrives faster.

- **High Availability:** the load is shared across many servers around the world. If one CDN server fails or traffic spikes, the others take over and the origin server stays protected.
- **Improved Security:** CDNs can absorb DDoS attacks, hide the origin server's real IP address, and handle encrypted (HTTPS) connections at the edge.

Section 9: When to Use CDNs and When to Hit the Origin Server

Use a CDN when:

- Delivering **static assets** like images, CSS files, JavaScript files, and video content.
- You need to ensure **high availability and performance** for users across different geographical locations.
- Reducing the **load on the origin server** is a priority.

Use the Origin Server when:

- Serving **dynamic content** that changes frequently or is personalized for individual users.
- Handling tasks that require **real-time processing** or access to up-to-date data.
- The application requires **complex server-side logic** that cannot be replicated or cached by a CDN.

Author & Copyright Notice

This document was made by **Mohammad Ali Joumaa**.

© 2026 Mohammad Ali Joumaa. All rights reserved. This document is copyrighted and may not be used, copied, reproduced, or distributed in any form without the express permission of the author.